Universidad del Valle de Guatemala
Ingeniería de Software 2
Sección 30
Ing. Lynette García



Sprint 9 - Proyecto “Sistema administrador para cadena de farmacias San Gabriel”

Josué Antonio Isaac García Barrera – 24918
José Manuel Sanchez Hernández - 24092
José Alberto Abril Suchite - 24585
Pablo André Orellana Mijangos - 20555
Andrés Esteban Ismalej González - 24005

Guatemala, 13 de octubre del 2026

1. Objetivos del Sprint

- Finalizar la funcionalidad del QR en el módulo de inventario, añadiendo seguridad.
- Registrar y mostrar el historial de cambios de precio de los lotes para el dueño y la administradora.
- Mejorar la búsqueda de productos tolerante a errores ortográficos con el algoritmo de Damerau-Levenshtein.
- Generar el resumen acumulado de ventas por producto, por período y sucursal.
- Refactorizar el manejo de errores y validaciones, el servicio de ventas y la gestión de contactos, para reducir la deuda técnica.
- Automatizar pruebas e integración continua con GitHub Actions y desplegar el incremento en producción.

2. Product Backlog

Lista de todas las historias de usuario y en qué sprint se completaron

Historia de Usuario	Descripción Corta	Prioridad	Sprint Completado
Doctor / Dueño			
Como doctor y dueño de la farmacia, quiero tener estadísticas de venta para maximizar las ganancias.	Dashboard con estadísticas y gráficos de ventas	Alta	Sprint 7, 8
Como doctor y dueño de la farmacia, quiero que se lleve un registro de las fechas de vencimiento para evitar pérdidas con productos caducados.	Control de fechas de vencimiento por lote	Alta	Sprint 4
Como doctor y dueño, quiero recibir alertas de stock mínimo para reabastecer productos antes de que se agoten y no perder ventas.	Alertas automáticas de stock bajo	Alta	Sprint 4

Como doctor y dueño, quiero generar informes de rentabilidad por sucursal para comparar el rendimiento de ambas farmacias.	Reportes financieros por sucursal	Alta	Sprint 8
Como doctor y dueño, quiero poder gestionar usuarios y permisos para controlar quién accede a cada función del sistema.	CRUD de usuarios con roles	Alta	Sprint 1
Como doctor y dueño, quiero poder crear, editar y eliminar sucursales para tener control de lo que se hace en ellas.	CRUD de sucursales y ciudades	Alta	Sprint 1, 2, 4
Como doctor y dueño, quiero poder tener registro de los proveedores y casas farmacéuticas asociadas.	CRUD de proveedores y casas	Alta	Sprint 3, 4
Dependiente (Caja)			
Como dependiente, quiero poder imprimir o enviar por correo tickets de venta para ofrecer opciones de comprobante a los clientes.	Generación y envío de tickets	Alta	Sprint 6
Como dependiente, quiero que el sistema muestre autocompletado mientras escribo para seleccionar productos más rápido.	Autocompletado en búsqueda	Alta	Sprint 7
Como dependiente, quiero generar cierre de caja automático por turno o día para entregar cuentas claras al siguiente turno.	Cierre de caja	Alta	Sprint 6, 8

Como dependiente, al identificar a un cliente frecuente, quiero ver su historial de compras y productos recurrentes para ofrecer una atención personalizada.	Historial de compras por cliente	Alta	Sprint 5
Como dependiente, quiero ver el inventario disponible de mi sucursal con detalles de lotes para saber qué tengo para vender.	Inventario por sucursal con lotes	Alta	Sprint 4
Como dependiente, quiero gestionar el carrito de compras: agregar, modificar cantidades y eliminar items, para revisar la venta antes de procesar el pago.	Gestión del carrito en POS	Alta	Sprint 5, 6
Como dependiente, quiero ver todos los lotes disponibles de un producto en el POS y poder seleccionar cuál vender, para gestionar mejor el inventario y evitar vender productos próximos a vencer sin opción.	Visualización en el POS	Alta	Sprint 7
Como dependiente, quiero que el sistema me advierta si intento vender más unidades de las disponibles en el lote seleccionado, para evitar sobreventas.	Manejo de errores en POS	Alta	Sprint 7
Como dependiente, quiero procesar pagos con tarjeta utilizando el celular como POS, para agilizar los cobros sin necesidad de datáfono adicional.	Pago con tarjeta	Alta	Sprint 6, 7

Administradora			
Como administradora, quiero que se unifiquen los ingresos de efectivo con los de tarjeta para facilitar las cuentas.	Unificación de pagos en venta	Alta	Sprint 5
Como administradora, quiero que haya una comparación automática del precio de compra y el de venta para agilizar el ingreso de datos.	Validación automática de precios	Media	Sprint 3
Como administradora, quiero poder registrar nuevos productos con todos sus datos, para tener un control exacto del inventario.	CRUD de productos y lotes	Alta	Sprint 3, 4, 6
Como administradora, quiero poder dar de baja productos vencidos para evitar conflictos.	Baja de productos vencidos	Alta	Sprint 4
Como administradora, quiero filtrar productos por categoría o combinaciones para localizar rápidamente cualquier artículo.	Filtros avanzados de búsqueda	Media	Sprint 3, 6
Como administradora, quiero consultar el historial de cambios de precio de un producto para decidir ajustes de manera informada.	Historial de precios	Media	Sprint 9
Como administradora, quiero consultar el historial y el resumen de cierres de caja por sucursal y método	Historial de cierres de caja	Alta	Sprint 8

de pago para supervisar las cuentas.			
Como administradora, quiero ver el resumen acumulado de ventas por producto de un período y sucursal, para registrar esas ventas en otro lugar.	Resumen acumulado de ventas por producto.	Media	Sprint 9
Enfermera			
Como enfermera, quiero verificar la fecha de vencimiento de un medicamento antes de venderlo para garantizar la seguridad del paciente.	Validación de vencimiento en venta	Alta	Sprint 5
Como enfermera, quiero que el sistema me sugiera cuándo un paciente podría necesitar su próxima dosis o servicio si es un cliente frecuente.	Sugerencias basadas en historial	Media	-
Como enfermera, quiero buscar productos por cualquier nombre (comercial o genérico) para encontrarlos rápidamente.	Búsqueda por nombre comercial/genérico	Alta	Sprint 3
Como enfermera, quiero que el sistema tolere errores ortográficos en la búsqueda para encontrar resultados aunque escriba mal el nombre del medicamento.	Tolerancia a errores ortográficos	Media	Sprint 8, 9
Como enfermera, quiero registrar un cliente frecuente con sus datos básicos para poder	CRUD de clientes frecuentes	Alta	Sprint 5, 6

identificar sus compras habituales.			
Laboratorio			
Como doctor y dueño, quiero un rol de laboratorista con acceso exclusivo al módulo de laboratorio para que el personal del laboratorio no vea la información de la farmacia.	Rol laboratorista y control de acceso	Media	Sprint 8
Como laboratorista, quiero registrar y editar pacientes para tener sus datos disponibles en cada visita.	Gestión de pacientes	Media	Sprint 8
Como laboratorista, quiero que cada paciente tenga un expediente con el historial de sus visitas para consultar sus atenciones anteriores.	Expediente y visitas	Media	Sprint 8
Como doctor y dueño, quiero que los expedientes no se eliminen sino que se anulen y que quede registro de cada cambio, para mantener la trazabilidad de la información médica.	Anulación y bitácora de cambios	Media	Sprint 8, Sprint 9
Como laboratorista, quiero adjuntar los resultados en PDF a cada visita para tenerlos guardados en el expediente del paciente.	PDF de resultados por visita	Media	

Como laboratorista, quiero consultar el catálogo de exámenes del laboratorio para orientar al paciente sobre los exámenes disponibles.	Catálogo de exámenes del laboratorio	Media	Sprint 9
Como paciente, quiero escanear el código QR impreso en mi PDF de resultados para ver la información de mi visita desde el celular.	Código QR en el PDF	Media	Sprint 8, 9

Vínculo del User Story Map:
https://miro.com/app/board/uXjVG7j97Wl=/?share_link_id=577092744581

Historias de Usuario incluidas en el Sprint 9

Historia de Usuario	Prioridad en este Sprint	Story Points
Catálogo de exámenes del laboratorio (laboratorista)	Alta	4.5
Código QR en el PDF (paciente)	Media	4.5
Historial de precios (administradora)	Alta	3.5
Anulación y bitácora de cambios (expedientes)	Alta	1.5
Tolerancia a errores ortográficos (enfermera)	Alta	3.5
Resumen acumulado de ventas por producto.	Media	3.0
Pruebas, refactorización, infraestructura y documentación	Alta	17.5

Tareas para este Sprint y asociación con historias de usuario

Historia de usuario asociada	Tarea	Prioridad
Catálogo de exámenes del laboratorio (laboratorista)	Base de datos: Catálogo de exámenes y registro de consultas públicas	Alta
	Backend: Catálogo de exámenes	Alta
	Frontend: Catálogo de exámenes	Alta
Código QR en el PDF (paciente)	Backend: Endurecer el endpoint público del QR	Media
	Frontend: Página pública del QR	Media
	Frontend: Revocar y regenerar QR	Media
Historial de precios (administradora)	Base de datos: Historial de cambios de precio	Alta
	Backend: Historial de cambios de precio	Alta
	Frontend: Historial de precios del producto	Alta
Anulación y bitácora de cambios (expedientes)	Backend: Historial de cambios del expediente	Alta
	Frontend: Historial de cambios del expediente	Alta
Tolerancia a errores ortográficos (enfermera)	Backend: Búsqueda tolerante con Damerau-Levenshtein	Alta
	Frontend: “¿Quiso decir…?” en la búsqueda del POS	Alta
Resumen acumulado de ventas por producto.	Backend: Resumen acumulado de ventas por producto	Media

	Frontend: Historial de ventas con resumen de productos acumulados	Media
Refactorización	Refactorización: Errores y validaciones centralizados	Alta
	Refactorización: VentaService fase 2 (webhooks y pagos POS)	Alta
	Refactorización: Consolidar correos y teléfonos	Alta
Infraestructura	Infraestructura: Integración continua con GitHub Actions	Alta
	Tests: Integración con base de datos real en el CI	Alta
	Infraestructura: Despliegue y pruebas en producción	Alta
Pruebas	Tests: Regresión de las refactorizaciones	Alta
	Tests: Unitarias automatizadas del backend	Alta
	Tests: Unitarias automatizadas del frontend	Alta
Documentación	Gestión: Crear tareas en Jira e iniciar Sprint	Alta
	Documentación: Métricas y reporte del Sprint	Alta
	Gestión: Revisión y video de demostración final	Alta
	LOGT individual	Alta

3. Sprint Backlog

Tarea	Descripción	Horas	Story Points	Responsable	Fecha probable de terminación
Gestión: Crear tareas en Jira e iniciar Sprint	Configurar el Sprint 9 en Jira con tareas y SP.	1	0.5	José Sanchez	03/10/2026
Infraestructura: Integración continua con GitHub Actions	Workflow en push y PR a develop que ejecute pruebas de backend (Jest con PostgreSQL de servicio) y frontend (Vitest), publique el reporte de cobertura y bloquee el merge si fallan. Documentar la herramienta para la guía.	4	2	José Sanchez	05/10/2026
Base de datos: Catálogo de exámenes y registro de consultas públicas	Agregar examen_laboratorio y una tabla para registrar cada consulta pública del QR (fecha e IP). Incluir seed y migración.	3	1.5	Andrés Ismalej	05/10/2026
Base de datos: Historial de cambios de precio	Registrar cada cambio de precio de compra o venta de un lote (valor anterior, valor nuevo, usuario y fecha) con su tabla y migración.	2	1	Jose Abril	05/10/2026
Backend:	DAO, servicio y	3	1.5	Andrés Ismalej	06/10/2026

Catálogo de exámenes	rutas. Consulta para laboratorista, dueño y administrador; crear, editar y desactivar solo para dueño y administrador (403 al laboratorista).				
Backend: Historial de cambios de precio	Endpoint para consultar el historial de precios con filtros por producto y fechas, solo para dueño y administrador.	2	1	Jose Abril	06/10/2026
Refactorización: Errores y validaciones centralizados	Crear AppError, asyncHandler y middleware común para express-validator y respuestas HTTP uniformes. Aplicarlo como piloto en Categoría y Ciudad, y separar app.js de server.js.	3	1.5	José Sanchez	06/10/2026
Backend: Endurecer el endpoint público del QR	Agregar un endpoint de datos de la visita (sin DPI), 404 genérico para token inválido o revocado, límite de peticiones (429), registro de cada consulta, y regenerar o revocar token.	4	2	Andrés Ismalej	07/10/2026
Backend:	Mantener el filtro	5	2.5	Josué García	07/10/2026

Búsqueda tolerante con Damerau-Levenshtein	por trigramas e índices GIN y reordenar los candidatos con Damerau-Levenshtein en una utilidad de Node, con normalización fonética del español y umbral de distancia según la longitud del término (1 edición hasta 4 letras, 2 hasta 8, 3 desde 9). Conservar tipo_coincidencia y relevancia en la respuesta. Incluir pruebas unitarias con casos de transposición, omisión, inserción, sustitución y errores fonéticos, y actualizar ProductoDAO.test.js.				
Frontend: Catálogo de exámenes	Pestaña Catálogo con buscador y filtro por área. Solo lectura para el laboratorista; crear, editar y desactivar para dueño y administrador.	3	1.5	Andrés Ismalej	07/10/2026
Backend: Historial de cambios del expediente	Endpoint de bitácora por expediente (solo dueño y administrador)	1	0.5	Jose Abril	07/10/2026

	que devuelva quién, cuándo y qué cambió, reutilizando el servicio de bitácora existente.				
Frontend: Historial de cambios del expediente	Pestaña “Historial de cambios” en la vista del expediente con quién, cuándo y qué cambió.	2	1	Jose Abril	08/10/2026
Refactorización: VentaService fase 2 (webhooks y pagos POS)	Dividir procesarWebhookRecurrente en validarEventoPago, determinarEstadoPago, registrarPagoRechazado y completarVentaPagada. Extraer el pago POS a su propio módulo y reemplazar estados y códigos repetidos por constantes.	4	2	Josué García	08/10/2026
Backend: Resumen acumulado de ventas por producto	Endpoint que devuelva el historial de ventas de un período y sucursal agrupado por producto, con cantidad total y monto acumulado, para facilitar el registro de las ventas en otro lugar. El formato	3	1.5	José Sanchez	08/10/2026

	exacto (columnas, agrupación y período) se afinará con el dueño.				
Frontend: Historial de precios del producto	Vista con tabla (y gráfica simple) de cambios de precio por producto, con filtro de fechas. Acceso para dueño y administrador.	3	1.5	Jose Abril	08/10/2026
Refactorización: Consolidar correos y teléfonos	Extraer la lógica común de contactos de sucursales, proveedores y casas farmacéuticas, y definir una sola vía entre ContactoSucursalService y Correo/TelefonoSucursalService.	3	1.5	Pablo Orellana	09/10/2026
Frontend: Página pública del QR	Ruta /resultados/:token fuera de ProtectedRoute, pensada para celular, con datos de la visita, botón para descargar el PDF y mensaje amable si el enlace no es válido.	3	1.5	Andrés Ismalej	09/10/2026
Frontend: Revocar y regenerar QR	Botones en la tarjeta del resultado para revocar o regenerar el	2	1	Andrés Ismalej	09/10/2026

	token, con confirmación.				
Frontend: “¿Quiso decir…?” en la búsqueda del POS	Mostrar sugerencia cuando solo hay coincidencias aproximadas o ninguna, resaltar la parte coincidente y conservar debounce, stock y selección de lotes.	2	1	Josué García	09/10/2026
Frontend: Historial de ventas con resumen de productos acumulados	Vista de historial de ventas que muestre el resumen de productos acumulados con su monto, con filtros de fecha y sucursal, pensada para copiar o transcribir los datos fácilmente. Se ajustará según lo que confirme el dueño.	3	1.5	José Sanchez	09/10/2026
Tests: Regresión de las refactorizaciones	Pruebas Jest que confirmen que ventas, webhooks, contactos y respuestas de error mantienen su comportamiento externo.	2	1	Pablo Orellana	09/10/2026
Tests: Unitarias automatizadas del backend	Jest y Supertest para catálogo, endpoint público, historial de precios, historial	2	1	Pablo Orellana	10/10/2026

	de cambios, resumen de ventas y componentes refactorizados, con casos exitosos, validaciones, errores y accesos denegados por rol.				
Tests: Unitarias automatizadas del frontend	Vitest y Testing Library para catálogo, página pública del QR, historial de precios, historial de ventas, “¿Quiso decir?” y controles de QR.	2	1	Pablo Orellana	10/10/2026
Tests: Integración con base de datos real en el CI	Supertest contra PostgreSQL, ejecutado en el workflow: resultado con QR, consulta pública, bitácora, historial de precios, resumen de ventas y búsqueda aproximada. Guardar el resultado como evidencia de integración continua.	3	1.5	José Sanchez	10/10/2026
Infraestructura: Despliegue y pruebas en producción	Desplegar el Sprint 9, aplicar migraciones, confirmar el volumen de PDF, definir la URL pública del QR en el servidor y actualizar el	1	1	Josué García	12/10/2026

	README.				
Documentación: Métricas y reporte del Sprint	Burndown, velocidad histórica, éxito del sprint (1 a 10), presupuesto y tiempo ejecutados, y listas de tareas terminadas, en proceso y no concluidas.	2	1	Josué García	13/10/2026
Gestión: Revisión y video de demostración final	Verificar consistencia entre Jira, repositorio e informe y grabar el video del incremento en producción.	2	1	José Sanchez	13/10/2026
LOGT - Jose Abril	Llenar el formulario LOGT con las tareas del sprint.	1	0.5	Jose Abril	13/10/2026
LOGT - Josué García	Llenar el formulario LOGT con las tareas del sprint.	1	0.5	Josué García	13/10/2026
LOGT - José Sanchez	Llenar el formulario LOGT con las tareas del sprint.	1	0.5	José Sanchez	13/10/2026
LOGT - Andrés Ismalej	Llenar el formulario LOGT con las tareas del sprint.	1	0.5	Andrés Ismalej	13/10/2026
LOGT - Pablo Orellana	Llenar el formulario LOGT con las tareas del sprint.	1	0.5	Pablo Orellana	13/10/2026

Calendario de planificación del sprint

Fecha de terminación posible	Tarea planificada
03/10/2026	Gestión: Crear tareas en Jira e iniciar Sprint
05/10/2026	Infraestructura: Integración continua con GitHub Actions
	Base de datos: Catálogo de exámenes y registro de consultas públicas
	Base de datos: Historial de cambios de precio
06/10/2026	Backend: Catálogo de exámenes
	Backend: Historial de cambios de precio
	Refactorización: Errores y validaciones centralizados
07/10/2026	Backend: Endurecer el endpoint público del QR
	Backend: Búsqueda tolerante con Damerau-Levenshtein
	Frontend: Catálogo de exámenes
	Backend: Historial de cambios del expediente
08/10/2026	Frontend: Historial de cambios del expediente
	Refactorización: VentaService fase 2 (webhooks y pagos POS)
	Backend: Resumen acumulado de ventas por producto
	Frontend: Historial de precios del producto
09/10/2026	Refactorización: Consolidar correos y teléfonos
	Frontend: Página pública del QR
	Frontend: Revocar y regenerar QR

	Frontend: “¿Quiso decir…?” en la búsqueda del POS
	Frontend: Historial de ventas con resumen de productos acumulados
	Tests: Regresión de las refactorizaciones
11/10/2026	Tests: Unitarias automatizadas del backend
	Tests: Unitarias automatizadas del frontend
	Tests: Integración con base de datos real en el CI
12/10/2026	Infraestructura: Despliegue y pruebas en producción
13/10/2026	Documentación: Métricas y reporte del Sprint
	Gestión: Revisión y video de demostración final
	LOGT individual

4. Incremento

a. Vínculo al repositorio de GitHub y Jira

GitHub: https://github.com/FarmaComGT/FarmaCom

Jira:
https://uvg-team-sw-1.atlassian.net/?continue=https%3A%2F%2Fuvg-team-sw-1.atlassian.net%2Fwelcome%2Fsoftware%3FprojectId%3D10000&atlOrigin=eyJpIjoiZTJmMTg0MTAwMjk5NDdlOWE0OTk5MTY1YjQ0YmRmZjQiLCJwIjoiamlyYS1zb2Z0d2FyZSJ9

Link del sistema: https://farmacom.website/login

Video del funcionamiento: https://youtu.be/tBUl6gBUPoA

b. Lista de funcionalidades terminadas

- Gestión: Crear tareas en Jira e iniciar Sprint

- Infraestructura: Integración continua con GitHub Actions
- Base de datos: Catálogo de exámenes y registro de consultas públicas
- Base de datos: Historial de cambios de precio
- Backend: Catálogo de exámenes
- Backend: Historial de cambios de precio
- Refactorización: Errores y validaciones centralizados
- Backend: Endurecer el endpoint público del QR
- Backend: Búsqueda tolerante con Damerau-Levenshtein
- Frontend: Catálogo de exámenes
- Backend: Historial de cambios del expediente
- Frontend: Historial de cambios del expediente
- Refactorización: VentaService fase 2 (webhooks y pagos POS)
- Backend: Resumen acumulado de ventas por producto
- Frontend: Historial de precios del producto
- Refactorización: Consolidar correos y teléfonos
- Frontend: Página pública del QR
- Frontend: Revocar y regenerar QR
- Frontend: "¿Quiso decir…?" en la búsqueda del POS
- Frontend: Historial de ventas con resumen de productos acumulados
- Tests: Regresión de las refactorizaciones
- Tests: Unitarias automatizadas del backend
- Tests: Unitarias automatizadas del frontend
- Tests: Integración con base de datos real en el CI
- Infraestructura: Despliegue y pruebas en producción
- Documentación: Métricas y reporte del Sprint
- Gestión: Revisión y video de demostración final
- LOGT individual.

c. Lista de tareas en proceso

Todas las tareas que se iniciaron se pudieron terminar.

d. Lista de tareas que se planificaron y no se pudieron concluir

No hubo ninguna tarea que se planificó y no se realizó.

5. Pruebas

a. Plan maestro de pruebas

Link del documento:
https://docs.google.com/document/d/1TthfMapmgS-_9l_BkEaE1219oeMiJ3_wHop8NRkWABg/edit?usp=sharing

b. Evidencia de implementación de las pruebas y resultados de las pruebas desarrolladas en el sprint

Link del video: **https://youtu.be/AA6wLv4BEeI**

c. Resultados de las pruebas

Durante el Sprint 9 se automatizaron dos tipos de pruebas:

1. Pruebas unitarias del backend y frontend.
2. Pruebas de integración y regresión entre la API y PostgreSQL.

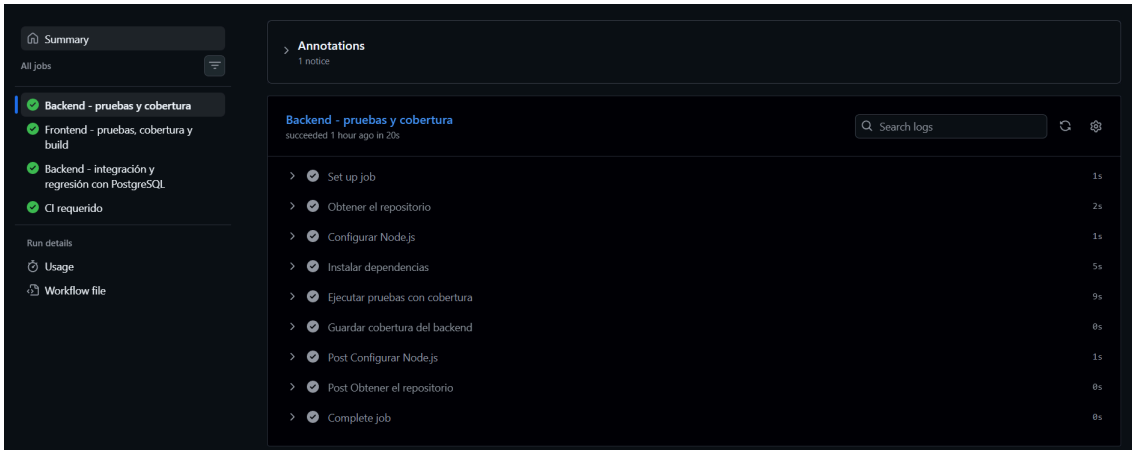
Las pruebas se ejecutaron automáticamente mediante GitHub Actions en los pull requests dirigidos a develop y en cada push realizado sobre dicha rama.

Pruebas unitarias del backend

El backend utiliza Jest para ejecutar las pruebas de controladores, servicios, DAO, middlewares y funciones auxiliares.

La ejecución más reciente produjo:

Resultado	Cantidad
Suites ejecutadas	101
Suites aprobadas	101
Pruebas ejecutadas	839
Pruebas aprobadas	839
Pruebas fallidas	0

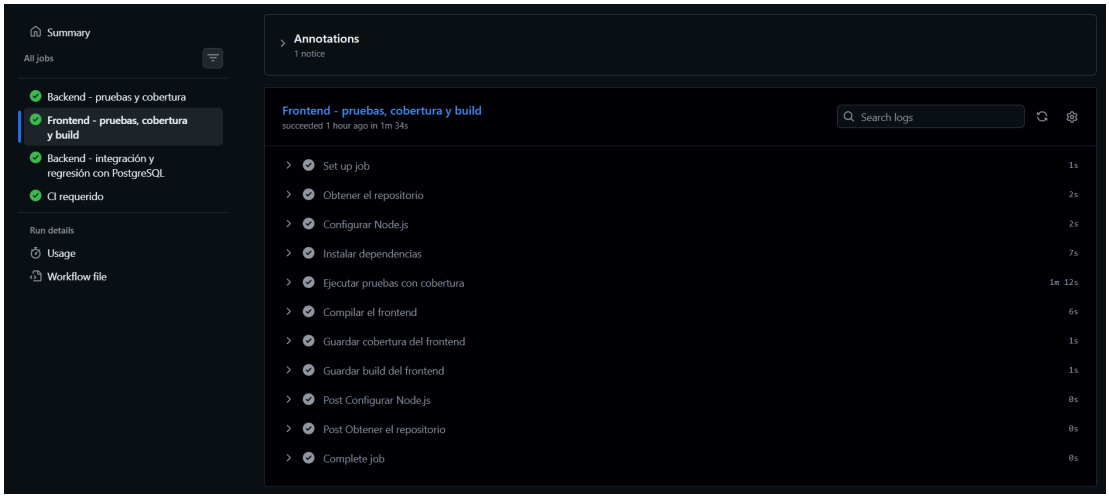


Pruebas unitarias del frontend

El frontend utiliza Vitest y el proveedor V8 para ejecutar pruebas y medir cobertura.

Resultado	Cantidad
Archivos de prueba ejecutados	98
Archivos aprobados	98
Pruebas ejecutadas	534
Pruebas aprobadas	534
Pruebas fallidas	0

Las pruebas cubren componentes, páginas, contextos, hooks y funciones auxiliares del frontend.



Pruebas de integración

Se implementaron pruebas de integración con componentes reales, como:

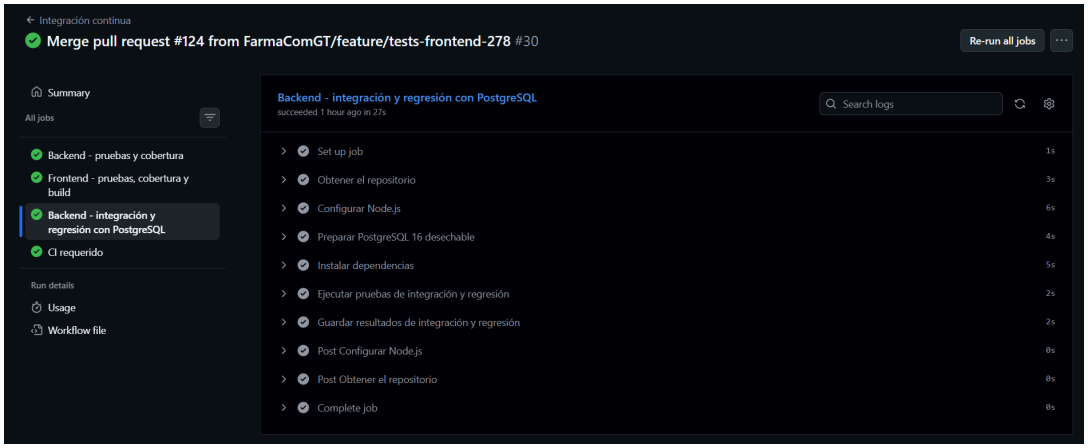
1. Inicio de sesión mediante ruta, controlador, servicio, DAO, bcrypt, JWT y PostgreSQL.
2. Creación transaccional de un paciente y su expediente.
3. Consulta mediante la API del paciente y expediente almacenados.

Las pruebas utilizan PostgreSQL 16 real y datos completamente ficticios. No se simulan la autenticación, los DAO ni la base de datos.

Pruebas de regresión

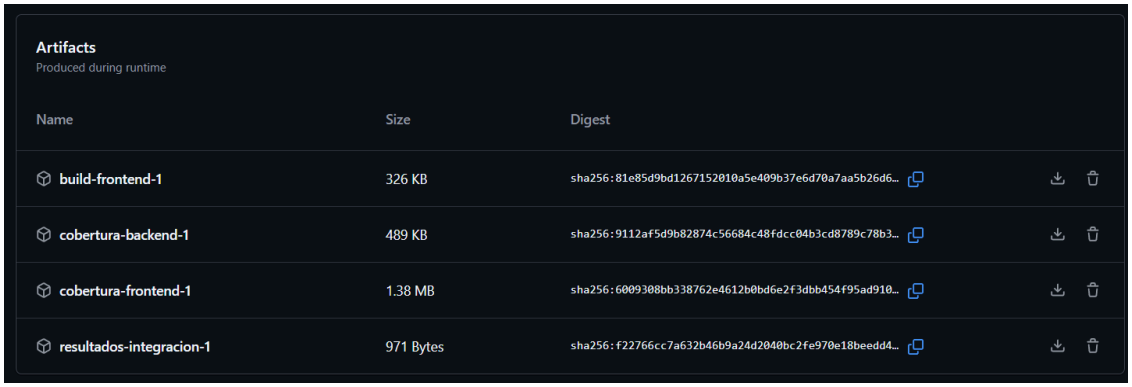
También se automatizaron tres comportamientos previamente implementados:

1. El login debe continuar generando una cookie HttpOnly.
2. Un laboratorista debe continuar recibiendo 403 al intentar utilizar funciones administrativas de farmacia.
3. La búsqueda de productos debe continuar siendo tolerante a mayúsculas y tildes.



Cobertura

Los reportes de cobertura se generan automáticamente en GitHub Actions y se conservan como artefactos durante 14 días.



Resumen general de pruebas

Área	Pruebas	Aprobadas	Fallidas
Backend unitario	839	839	0
Frontend unitario	534	534	0
Integración y regresión	6	6	0
Total	1,379	1,379	0

6. Integración Continua

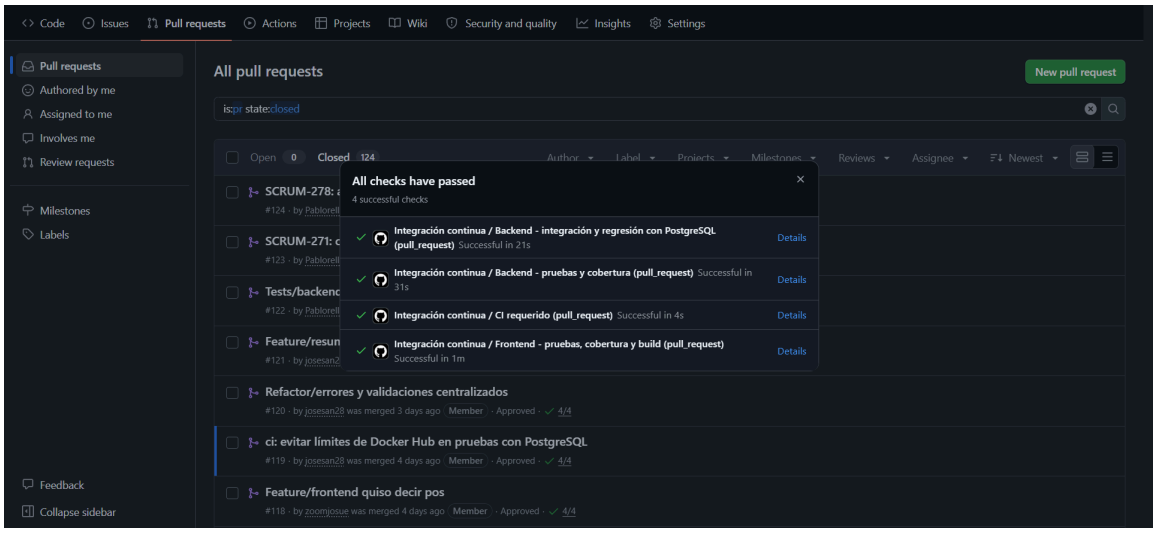
a. Descripción del proceso

Durante el Sprint 9 se implementó integración continua mediante GitHub Actions. El equipo trabajó en ramas independientes y utilizó pull requests para integrar cambios en develop.

El proceso fue el siguiente:

1. Cada integrante realizó cambios en una rama separada.
2. Los cambios se dividieron en commits pequeños y significativos.
3. Se abrió un pull request hacia develop.
4. GitHub Actions ejecutó automáticamente las pruebas del backend.
5. GitHub Actions ejecutó las pruebas y el build del frontend.
6. Se preparó una base PostgreSQL 16 temporal.
7. Se ejecutaron las pruebas de integración y regresión.
8. Se publicaron los reportes como artefactos.
9. El job CI requerido comprobó el resultado de todos los controles.
10. El pull request quedó disponible para revisión e integración.

El workflow también se ejecuta después de realizar un push a develop y puede iniciarse manualmente mediante workflow_dispatch.



b. Herramientas utilizadas

GitHub Actions

Se utilizó para ejecutar automáticamente las validaciones ante pull requests y cambios en develop.

Jest

Se utilizó para las pruebas unitarias y la cobertura del backend.

Vitest

Se utilizó para las pruebas unitarias y la cobertura del frontend.

Supertest

Se utilizó para enviar solicitudes HTTP reales a la aplicación Express durante las pruebas de integración.

PostgreSQL 16

Se utilizó como base de datos real y temporal para las pruebas de integración. La base se crea dentro del runner y se elimina junto con la máquina virtual al terminar la ejecución.

Vite

Se utilizó para comprobar que el frontend puede generar correctamente un build de producción.

c. Controles del workflow

Job	Función
Backend - pruebas y cobertura	Ejecuta las pruebas unitarias y genera cobertura del backend
Frontend - pruebas, cobertura y build	Ejecuta las pruebas, genera cobertura y compila el frontend
Backend - integración y regresión con PostgreSQL	Prepara PostgreSQL y ejecuta las seis pruebas
CI requerido	Comprueba que todos los jobs obligatorios hayan aprobado

d. Evidencia de una semana de integración continua

El workflow base fue fusionado en develop el 5 de octubre de 2026. A partir de esa fecha se ejecutaron validaciones automáticas durante los pull requests y los pushes realizados por el equipo.

En el siguiente link se pueden ver los commits en donde se empezó a utiliza la integración continua:

https://github.com/FarmaComGT/FarmaCom/commits/develop/?after=488ec2410d4ef96ac7a4cd4e8540e54e2e811217+34

e. Reflexión sobre el rendimiento del equipo

La integración continua permitió detectar errores antes de incorporar los cambios a develop. Las pruebas automáticas proporcionaron retroalimentación sobre el backend, el frontend, el build y la interacción con PostgreSQL.

El proceso también mejoró la trazabilidad, porque cada resultado quedó asociado a un commit y a una ejecución específica. Los artefactos permitieron conservar los reportes de cobertura y los resultados de integración.

Como aspecto positivo, el equipo pudo identificar áreas con cobertura insuficiente y agregar pruebas dirigidas a contextos, hooks, utilidades, páginas, controladores, servicios y DAO. En el frontend, statements aumentó de 62.46 % a 80.72 %, mientras que lines aumentó de 63.60 % a 81.77 %. También se consiguió una mayor confianza en los cambios realizados al observar que las pruebas se ejecutaban sin fallos, lo que permite revisar de mejor manera y avanzar más rápido.

Como aspecto negativo, la configuración inicial requirió tiempo para separar la aplicación Express, preparar datos ficticios y estabilizar PostgreSQL en GitHub Actions. También se debe investigar sobre el límite de uso que tiene GitHub Actions para evitar que la implementación actual de CI consuma todo el tiempo que se da en el plan que se tiene.

Para mejorar el proceso en el siguiente sprint, el equipo debe:

- Revisar los resultados de CI después de cada push.
- Mantener cambios pequeños.
- Corregir inmediatamente los workflows fallidos.
- Distribuir los commits de manera constante durante el sprint.
- Agregar pruebas enfocadas en ramas condicionales y respuestas de error.

Comparación con el sprint anterior

Uno de los principales beneficios fue la mejora en la revisión de los pull requests, ya que las pruebas del backend, del frontend y las pruebas de integración y regresión se ejecutaron automáticamente antes de incorporar los cambios a la rama develop. De esta manera, los revisores pudieron comprobar con mayor facilidad si una modificación cumplía con las validaciones establecidas y si afectaba alguna funcionalidad existente.

La automatización también permitió que el equipo avanzara con mayor rapidez. Antes, parte del proceso dependía de ejecutar las pruebas manualmente y comunicar sus resultados, lo cual requería más tiempo y podía ocasionar que algunos problemas se descubrieran después de integrar los cambios. Con el flujo de integración continua, cada actualización generó resultados claros, facilitando la identificación y corrección de errores antes de aceptar un pull request.

Además, la generación automática de reportes de cobertura y resultados de pruebas proporcionó evidencia objetiva sobre el estado del sistema. Esto mejoró la trazabilidad del trabajo realizado y permitió verificar que tanto el código nuevo como las funcionalidades existentes continuaran funcionando correctamente. Las pruebas de regresión también ayudaron a comprobar que las correcciones aplicadas no introdujeran nuevos errores en otras partes del proyecto.

En general, con respecto al sprint anterior, el equipo contó con un proceso más ordenado, confiable y eficiente. La integración continua disminuyó el riesgo de incorporar cambios defectuosos, facilitó la colaboración entre los integrantes y permitió dedicar más tiempo al desarrollo y a la revisión de la calidad del código. Aunque todavía pueden realizarse mejoras al flujo en próximos sprints, la automatización implementada representó un avance importante en la estabilidad del proyecto y en la velocidad con la que se desarrollaron e integraron las nuevas funcionalidades.

7. Refactorización

a. Deuda técnica identificada

Deuda identificada	Evidencia en el proyecto	Consecuencia	Prioridad
Componentes de interfaz demasiado grandes	ProductoFormModal.jsx tiene aproximadamente 530 líneas,	Dificulta comprender, modificar y probar cada responsabilidad de forma	Alta

	LoteFormModal.jsx 505 líneas y PuntoVenta.jsx 417 líneas.	independiente.	
Servicio de ventas con demasiadas responsabilidades	VentaService.js tiene aproximadamente 468 líneas y administra validaciones, cálculos monetarios, inventario, ventas en efectivo, pagos con POS y webhooks.	Aumenta el acoplamiento y el riesgo de afectar ventas o existencias al modificar la integración de pagos.	Crítica
Métodos extensos y condicionales complejos	procesarWebhookRe currente concentra validación, cambio de estados, rechazo de pagos, creación de ventas y actualización del inventario.	Hace más difícil seguir el flujo y probar cada escenario por separado.	Crítica
Código duplicado en la gestión de contactos	Los servicios de correos y teléfonos de casas farmacéuticas, proveedores y sucursales repiten operaciones de creación, consulta, actualización, eliminación y manejo de errores.	Una corrección debe aplicarse en varios lugares y existe el riesgo de obtener comportamientos diferentes.	Alta
Duplicación en controladores	Controladores como CategoriaController.js y CiudadController.js repiten la validación de solicitudes, bloques try/catch y construcción de respuestas de error.	Incrementa el volumen de código y dificulta mantener respuestas uniformes en la API.	Media
Responsabilidades mezcladas en las rutas	CasaFarmaceuticaRo utes.js contiene acceso directo a la base de datos para consultar	La ruta queda acoplada a PostgreSQL y rompe la separación entre rutas, controladores,	Alta

	proveedores asociados.	servicios y DAO.	
Validaciones extensas dentro de archivos de rutas	ProductoRoutes.js tiene aproximadamente 288 líneas y reúne una gran cantidad de reglas de validación junto con la declaración de endpoints.	Reduce la legibilidad de las rutas y dificulta reutilizar las mismas reglas.	Media
Uso repetido de cadenas y valores de control	Estados como pagado, fallido, cancelado, rechazado y códigos HTTP aparecen directamente en diferentes condiciones.	Un error de escritura puede generar comportamientos inconsistentes.	Media
Manejo de errores no centralizado	Cada controlador decide individualmente cómo transformar una excepción en una respuesta HTTP. Algunos servicios asignan error.status, mientras que otros lanzan errores simples.	La API puede devolver respuestas distintas para situaciones equivalentes.	Alta
Automatización de calidad incompleta	Existen pruebas unitarias y de integración, pero no hay umbrales mínimos de cobertura, scripts de análisis estático ni un flujo de integración continua en el repositorio.	Una refactorización podría incorporarse sin detectar regresiones o problemas de estilo.	Alta
Convenciones y mensajes inconsistentes	Hay diferencias de formato, nombres de módulos con mayúsculas y minúsculas distintas	Reduce la uniformidad del código y afecta la calidad de los mensajes mostrados	Baja

	y mensajes sin algunas tildes, por ejemplo “sesion”, “invalido” o “esta inactivo”.	al usuario.	

No se considera necesaria una reescritura completa de FarmaCom. La arquitectura actual ya separa gran parte del backend en rutas, controladores, servicios y DAO, y además existen pruebas para funciones importantes. Por ello, la opción adecuada es una refactorización incremental.

b. Áreas que se necesitan refactorizar

i. Procesamiento de ventas y pagos

Es el área de mayor prioridad porque interviene directamente en el cobro, la actualización de existencias y el registro de transacciones.

Se refactorizará VentaService.js, especialmente los procesos crearVenta, crearPagoPOS y procesarWebhookRecurrente. El objetivo será separar:

- Validación de acceso a sucursales.
- Preparación y cálculo de la venta.
- Conversión y validación de montos.
- Descuento de existencias.
- Registro de detalles de venta.
- Procesamiento de pagos en efectivo.
- Procesamiento de pagos con tarjeta.
- Interpretación de eventos recibidos por webhook.
- Actualización de estados del pago.

También se extraerá la lógica repetida que crea los detalles de la venta y descuenta las existencias de cada lote.

ii. Formulario de productos y lotes

ProductoFormModal.jsx y LoteFormModal.jsx administran simultáneamente estado, validaciones, cálculos, carga de catálogos, presentación de campos y operaciones relacionadas.

Estos componentes se dividirán en secciones más pequeñas y reutilizables. La lógica del formulario se trasladará a hooks específicos, mientras que el componente principal conservará la coordinación y presentación del modal.

iii. Punto de venta

PuntoVenta.jsx controla el carrito, los límites de existencias, clientes, cobros, pagos con POS, consultas periódicas del estado del pago y diversos modales.
Se separará la lógica en elementos como:

- Hook para la administración del carrito.
- Hook para el proceso de cobro.
- Hook para el seguimiento del pago con POS.
- Función para construir el objeto enviado a la API.
- Componentes responsables de los avisos y estados del pago.

iv. Gestión de correos y teléfonos

Se consolidará la lógica común de los contactos de sucursales, proveedores y casas farmacéuticas. No se eliminarán las diferencias propias de cada entidad, pero se extraerán validaciones, manejo de errores y operaciones repetidas.

También se revisará la coexistencia de ContactoSucursalService con CorreoSucursalService y TelefonoSucursalService, con el propósito de establecer una única forma de administrar estos contactos.

v. Controladores, validaciones y errores

Se extraerá un middleware común para:

- Procesar los resultados de express-validator.
- Capturar errores asíncronos.
- Convertir los errores de negocio en respuestas HTTP uniformes.
- Evitar la repetición de bloques try/catch.

Las validaciones extensas de productos, lotes, proveedores y casas farmacéuticas se trasladarán a módulos independientes.

vi. Separación de responsabilidades en rutas

La consulta SQL ubicada en CasaFarmaceuticaRoutes.js se moverá a la capa DAO. La comprobación de existencia y las reglas del caso de uso se colocarán en el servicio, mientras que el controlador únicamente recibirá la solicitud y devolverá la respuesta.

Además, la creación y el inicio del servidor se separarán, por ejemplo, en app.js y server.js. Esto permitirá importar la aplicación en las pruebas sin abrir un puerto.

vii. Pruebas y controles automáticos

Antes de modificar los módulos críticos se completarán pruebas de caracterización que documenten su comportamiento actual. Posteriormente se incorporarán:

- Pruebas unitarias de los métodos extraídos.
- Pruebas de integración de ventas, pagos, inventario y webhooks.
- Pruebas de los formularios y hooks extraídos.
- Umbrales mínimos de cobertura.
- Análisis estático y formato automático.
- Ejecución automática de las pruebas antes de integrar cambios.

c. Técnicas a utilizar

Técnica	Aplicación concreta
Extraer método	Dividir procesarWebhookRecurrente en métodos como validarEventoPago, determinarEstadoPago, registrarPagoRechazado y completarVentaPagada.
Extraer clase o módulo	Separar de VentaService el procesamiento de pagos POS y la coordinación de webhooks.
Mover método	Trasladar el acceso a datos de CasaFarmaceuticaRoutes.js hacia el DAO correspondiente y mover las reglas del caso de uso al servicio.
Extraer componente	Dividir los formularios de productos y lotes en secciones pequeñas con responsabilidades específicas.
Extraer hook	Mover el estado y la lógica de formularios, carrito y seguimiento del pago a hooks reutilizables.

Eliminar código duplicado	Unificar la lógica común de correos, teléfonos, validaciones y respuestas de los controladores.
Reemplazar valores mágicos por constantes	Definir constantes para estados de pagos, roles, métodos de pago, límites monetarios y códigos utilizados repetidamente.
Encapsular errores	Crear un tipo de error de aplicación que contenga el mensaje, el código HTTP y, cuando corresponda, detalles de validación.
Simplificar condicionales	Sustituir cadenas extensas de condiciones de estados de pago por mapas de estados o funciones especializadas.
Renombrar elementos	Uniformar nombres de archivos, variables y funciones para que expresen claramente su responsabilidad.
Separar consulta de modificación	Mantener separados los métodos que únicamente consultan información de aquellos que actualizan ventas, pagos o existencias.

d. Tareas de refactorización llevadas a cabo en el Sprint 9

Tarea	Descripción	Problema que atiende	Acción realizada	Resultado
Refactorización: Errores y validaciones centralizados	Mecanismo común de manejo de errores y validaciones del backend, aplicado primero como piloto y luego extendido.	Manejo de errores no centralizado; try/catch y validaciones repetidos en controladores; aplicación y arranque del servidor en el mismo archivo.	Se crearon AppError, asyncHandler, validateRequest, errorHandler y notFoundHandler. Se aplicó en categorías y ciudades, y se extendió a presentaciones, sucursales, clientes, promociones, proveedores, casas farmacéuticas y	Respuestas HTTP uniformes en 15 controladores. Las pruebas importan la aplicación sin abrir un puerto y hay pruebas de manejo de errores y privacidad.

			contactos. Se separó app.js de server.js y se evitó exponer datos internos en las respuestas de error. Técnicas: encapsular errores, eliminar código duplicado, extraer módulo.	
Refactorización: VentaService fase 2 (webhooks y pagos POS)	Dividir el procesamiento de webhooks y aislar el pago POS del servicio de ventas.	Servicio con demasiadas responsabilidades, método de webhook extenso con condicionales complejos y cadenas repetidas.	Se dividió procesarWebhookRecurrente en validarEventoPago, determinarEstadoPago, registrarPagoRechazado y completarVentaPagada. El pago POS pasó a VentaPagoPOSService.js. Estados, métodos, códigos y moneda pasaron a VentaPagoConstantes.js. Técnicas: extraer método, extraer módulo, reemplazar valores mágicos.	VentaService.js bajó de unas 468 líneas a 142. Hay pruebas unitarias del nuevo módulo.
Refactorización: Consolidar correos y teléfonos	Unificar la lógica común de contactos de sucursales, proveedores y casas farmacéuticas.	Código duplicado en la gestión de contactos, con riesgo de comportamientos distintos.	Se centralizó el manejo de errores y las validaciones en los 7 controladores y servicios de contactos. [Completar: la	Menos código repetido en contactos y pruebas de manejo de errores de contactos.

			vía única entre ContactoSucursalService y CorreoSucursalService/TelefonoSucursalService.]	

e. Estado de la deuda técnica después del Sprint 9 y cómo se disminuye con lo realizado

Deuda identificada	Prioridad	Estado	Evidencia
Servicio de ventas con demasiadas responsabilidades	Crítica	Resuelta	VentaService.js de 142 líneas, con módulos de preparación, cálculo monetario, detalle y pago POS.
Métodos extensos y condicionales complejos	Crítica	Resuelta	procesarWebhookRecurrente dividido en cuatro funciones.
Código duplicado en la gestión de contactos	Alta	Resuelta	Refactorización de los 7 servicios y controladores de contactos.
Responsabilidades mezcladas en las rutas	Alta	Resuelta	La ruta de proveedores de una casa ya pasa por controlador, servicio y DAO.
Manejo de errores no centralizado	Alta	Parcial	15 de 30 controladores migrados; faltan Auth, Caja, Inventario, Lote, Paciente, Producto, Reporte, Usuario, Venta y otros.
Duplicación en controladores	Media	Parcial	Misma situación: los controladores migrados ya no repiten try/catch.

Uso repetido de cadenas y valores de control	Media	Parcial	Estados y métodos de pago ya son constantes. Siguen códigos HTTP literales en los controladores no migrados.
Automatización de calidad incompleta	Alta	Resuelta	Ya hay CI con pruebas, cobertura e integración con PostgreSQL.
Validaciones extensas dentro de rutas	Media	Pendiente	ProductoRoutes.js sigue con 291 líneas.
Componentes de interfaz demasiado grandes	Alta	Pendiente	Planificado para el Sprint 10.

Las refactorizaciones del sprint atacaron primero las deudas críticas: el procesamiento de pagos y webhooks, que interviene directamente en el cobro y el inventario. VentaService.js pasó de unas 468 a 142 líneas y los webhooks quedaron divididos en funciones con una sola responsabilidad, por lo que cada escenario se prueba por separado y un cambio en la integración de pagos pone menos en riesgo las ventas y las existencias. La centralización de errores y validaciones uniformó las respuestas de la API en 15 controladores y eliminó bloques try/catch repetidos. La consolidación de contactos redujo el código duplicado en sucursales, proveedores y casas farmacéuticas. Como los cambios fueron incrementales, se respaldaron con pruebas de regresión y con el CI en GitHub Actions, que ejecuta las pruebas de backend y frontend y las de integración con PostgreSQL en cada push y pull request a develop. La deuda restante pasa al Sprint 10.

f. Plan de refactorización

Momento	Actividad
Sprint 9	Continuar con la refactorización de ventas y pagos, eliminar la duplicación en la gestión de correos y teléfonos, y agregar pruebas para comprobar que el comportamiento funcional se mantiene.
Sprint 10	Refactorizar los componentes grandes del frontend, principalmente los formularios de

	productos y lotes y la lógica de PuntoVenta.jsx. También se deben migrar los controladores restantes al manejo centralizado y extraer las validaciones de ProductoRoutes.
Cierre del sprint 10	Ejecutar todas las pruebas del backend y frontend, verificar los flujos críticos de ventas e inventario y documentar la deuda solucionada y la que quede pendiente.
Entrega y presentación final	Realizar una verificación integral del sistema, presentar evidencias de las mejoras obtenidas y demostrar que la refactorización no modificó el comportamiento externo de FarmaCom. En esta etapa no se planifican cambios estructurales grandes, salvo correcciones indispensables encontradas durante la verificación.

La refactorización se realiza de manera incremental. Antes de cada cambio se asegura el comportamiento actual mediante pruebas, después de cada modificación se ejecutan las pruebas del módulo afectado y, antes de integrar los cambios, se verifica el sistema completo. Esta distribución permite pagar primero la deuda relacionada con las operaciones más críticas y dejar el sistema estable para la presentación final.

8. Ejecución de presupuesto

a. Resumen del cálculo de costos y tiempo

El costo total estimado para la ejecución del proyecto se mantiene en Q100,890.00, tomando como base un esfuerzo estimado de 2,242 horas-hombre distribuido entre un equipo de 5 integrantes a lo largo de 30 semanas. Actualmente, el proyecto se encuentra en la etapa de cierre del Sprint 9 de un total de 10 sprints, por lo que se registra un avance del 90% tanto en la ejecución del tiempo como del presupuesto planificado.

Adicionalmente, se contemplan los costos complementarios correspondientes al primer año de operación, los cuales ascienden a Q30,033.50. Considerando que la cadena de farmacias San Gabriel no posee un equipo técnico propio, opera con tres sucursales y requiere mantener un sistema centralizado con soporte continuo, se determinó viable el modelo combinado de SaaS y una tarifa única de implementación. Bajo este modelo, el costo estimado para el cliente durante el primer año es de Q28,000.00.

b. Porcentaje gastado hasta el momento del costo total calculado

De acuerdo con la planificación financiera, al completar el Sprint 9 se ha ejecutado aproximadamente el 90% del presupuesto total. Esto representa un gasto acumulado de Q90,801.00 de los Q100,890.00 presupuestados inicialmente.

El 10% restante, equivalente a Q10,089.00, corresponde al último sprint (Sprint 10) y a las actividades finales necesarias para la conclusión, el despliegue y la entrega formal del proyecto.



c. Porcentaje tiempo consumido del tiempo total estimado

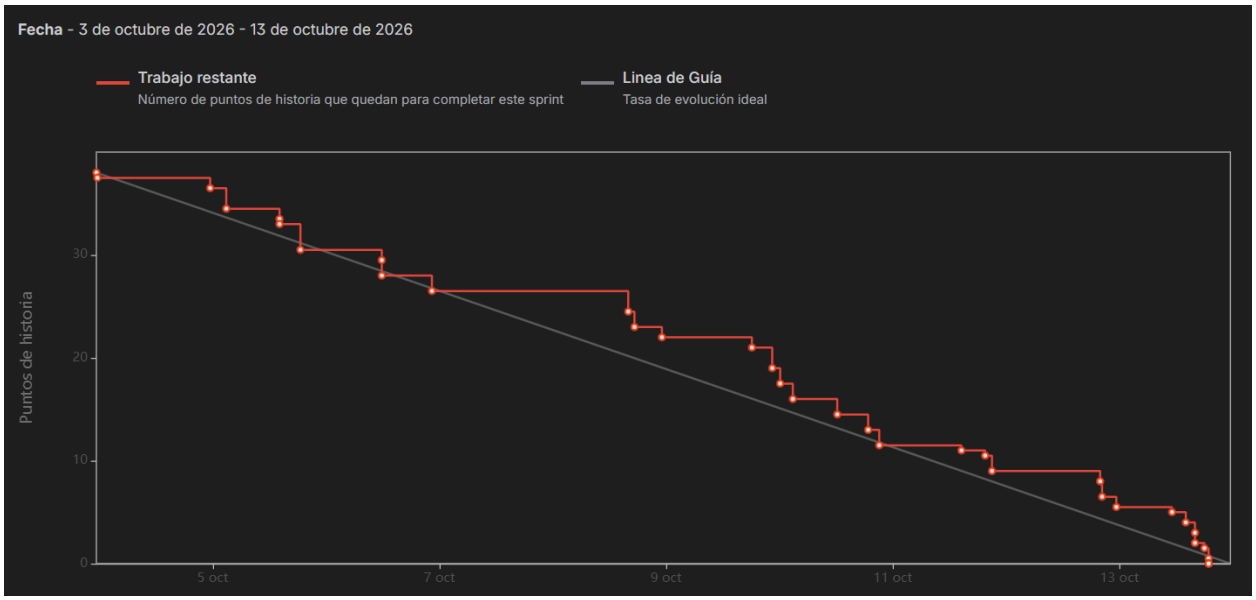
De las 30 semanas contempladas originalmente para el desarrollo del proyecto, se han utilizado 27 semanas, lo que representa un 90% del tiempo total planificado. En términos de esfuerzo, esto corresponde a 2,017.8 horas-hombre de las 2,242 estimadas al inicio.

Actualmente, tras la entrega del Sprint 9, resta únicamente una semana, equivalente al 10% del tiempo estimado, para completar el último sprint y realizar la presentación final del proyecto.

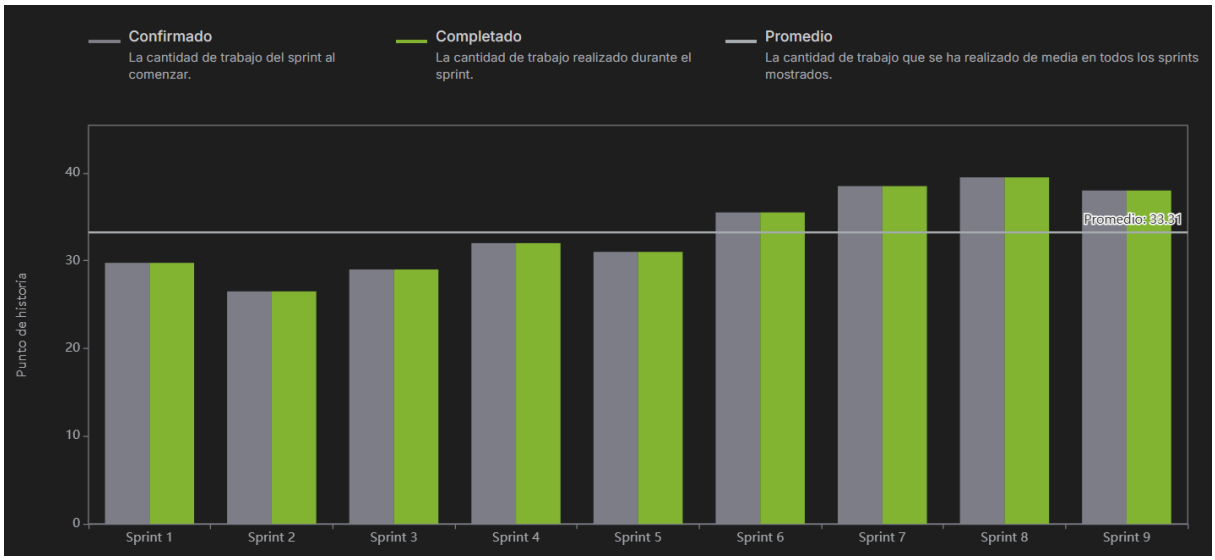


9. Resultados del Sprint

a. Gráfico Burndown



b. Métrica de velocidad



Sprint	Confirmado	Completado
Sprint 1	29.75	29.75
Sprint 2	26.5	26.5
Sprint 3	29	29
Sprint 4	32	32
Sprint 5	31	31
Sprint 6	35.5	35.5
Sprint 7	38.5	38.5
Sprint 8	39.5	39.5
Sprint 9	38	38

c. Indicador numérico del éxito del sprint y su justificación

Durante el Sprint 9 se completaron los 38 story points comprometidos, alcanzando una efectividad del 100% respecto al trabajo planificado. El equipo mantuvo el promedio y no se redujo el alcance en ninguna de las nueve iteraciones del proyecto. La velocidad de 38 SP se ubica aproximadamente un 14% por encima del promedio general.

Se utilizó la misma fórmula aplicada en sprints anteriores:

Éxito del Sprint = (0.7 × % de cumplimiento de SP) + (0.3 × factor de distribución del trabajo)

Donde el % de cumplimiento de SP corresponde a la proporción de story points completados respecto a los comprometidos, y el factor de distribución del trabajo evalúa qué tan uniforme fue el avance según el burndown.

Para el Sprint 9, el cumplimiento fue del 100%. El burndown muestra un avance sostenido durante los primeros cuatro días, con un trabajo restante cercano a la línea guía. A partir del 7 de octubre se observa una pausa de aproximadamente un día y medio y desde ese punto el trabajo restante se mantuvo por encima de la línea guía. Se observa además un segundo período de estancamiento entre el 11 y el 12 de octubre, seguido de una descarga concentrada en las últimas 36 horas, en las que se cerraron los puntos restantes que son el despliegue en producción, métricas, reporte, revisión final y formularios LOGT. Por ello se estima un factor de distribución de 0.85.

Éxito = (0.7 × 1.0) + (0.3 × 0.85) = 0.96, lo que equivale a 9.6/10.

d. Discusión del éxito del sprint basado en las métricas

Las métricas confirman que el Sprint 9 fue un sprint exitoso en cumplimiento. Se completaron los 38 SP comprometidos, con un 100% de cumplimiento. La velocidad es 1.5 SP menor que la del Sprint 8, pero como los puntos confirmados y completados coinciden, esta diferencia refleja una planificación ligeramente mejor. Las columnas de trabajo confirmado y completado coinciden en los nueve sprints, y el promedio del proyecto subió a 33.31 SP, tomando en cuenta también que este sprint estuvo por encima del mismo, lo que indica que el avance fue significativo.

Conviene señalar que de los 38 SP, 17.5 corresponden a pruebas, refactorización, infraestructura y documentación, y los 20.5 restantes a funcionalidad nueva, como catálogo de exámenes, QR, historial de precios, bitácora de expedientes, búsqueda tolerante a errores y resumen de ventas y su valor se refleja en resultados medibles:

Pruebas automatizadas: 1,379 pruebas, 839 de backend, 534 de frontend y 6 de integración y regresión. Todas fueron aprobadas y ejecutadas automáticamente en GitHub Actions.

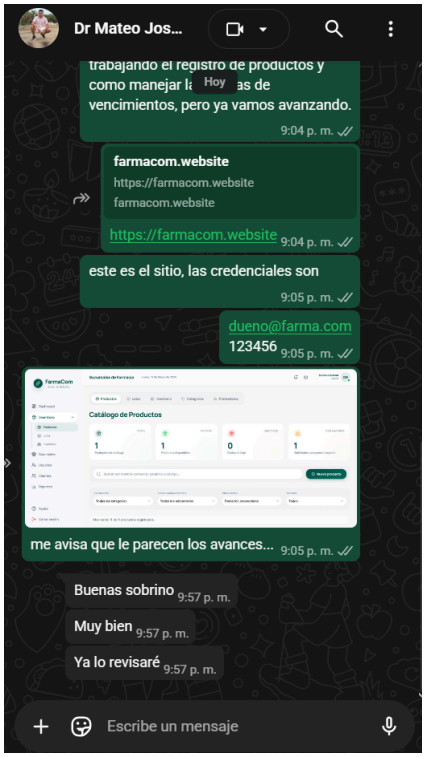
Cobertura del frontend: statements subió de 62.46% a 80.72% y lines de 63.60% a 81.77%.

Deuda técnica: VentaService.js bajó de unas 468 a 142 líneas y se resolvieron las deudas críticas de ventas y webhooks. Se migraron 15 de 30 controladores al manejo centralizado de errores y la deuda restante pasa al Sprint 10.

El trabajo se acumuló hacia el final en las pruebas de integración, el despliegue y la documentación dependían de que las funcionalidades y refactorizaciones estuvieran integradas en develop y el cierre se concentró en las últimas horas. Esto coincide con la mejora identificada en la reflexión de integración continua que fue en distribuir los commits de manera constante durante el sprint y revisar los resultados del CI después de cada push.

En conjunto, las métricas permiten concluir que el Sprint 9 cumplió el 100% de lo comprometido, incorporó la integración continua y elevó la cobertura de pruebas, pero que para el Sprint 10 conviene repartir mejor las tareas de cierre a lo largo del sprint y no dejarlas para los últimos días. El proyecto se encuentra al 90% del tiempo y presupuesto planificados, por lo que el último sprint debe enfocarse en estabilizar el sistema y preparar la entrega final.

e. Evidencias de muestra del incremento desarrollado a usuarios finales y/o product owner y comentarios al respecto



El product owner indicó que le gustó el historial de ventas y señaló que la pestaña agrupada por producto es justo de lo que más necesitaba. El historial de cambios de precio le pareció que añade mucha más información y seguridad en cuanto al módulo de inventario. Le pareció interesante que pueda filtrarse por cambios del precio de venta y de compra.

En cuanto al módulo de inventario, consideró que ya está finalizado. Indicó que algunas de las funcionalidades que se planificaron inicialmente ya no serán utilizadas, por lo que simplemente es cuestión de quitarlas del proyecto. Esto se realizará durante el próximo sprint.

En cuanto a lo visual, sigue considerando que es atractivo y la experiencia es buena en general. Hizo mención sobre ciertos botones que se podrían cambiar en la pantalla de inventario, pero por lo demás, todo bien.

f. Retrospectiva del sprint

Se identificó como principal aspecto positivo que el equipo cerró el 100% de los 38 story points comprometidos y que el burndown llegó a cero al final del sprint. Además de la funcionalidad nueva del catálogo de exámenes, QR endurecido, historial de precios, bitácora de expedientes, búsqueda tolerante a errores con Damerau-Levenshtein y resumen acumulado de ventas, el sprint incorporó prácticas de ingeniería que el proyecto no tenía. La más importante fue la integración continua con GitHub Actions, que desde su fusión en develop el 5 de octubre ejecuta automáticamente las pruebas de backend y frontend, el build y las pruebas de integración con PostgreSQL en cada pull request y en cada push.

También se valoró de forma favorable el efecto de la automatización sobre la calidad. Se llegó a 1,379 pruebas, todas aprobadas, y la cobertura del frontend subió de 62.46% a 80.72% en statements y de 63.60% a 81.77% en lines. El equipo consideró útil que el CI detectara errores antes de llegar a develop y que cada resultado quedara asociado a un commit y a una ejecución específica, lo que mejoró la trazabilidad y la confianza al integrar cambios.

Como aspectos a mejorar, se identificó que el trabajo volvió a concentrarse hacia el final del sprint. El burndown muestra períodos de estancamiento y una descarga concentrada en las últimas horas, en las que se cerraron el despliegue, las métricas, el reporte, la revisión final y los formularios LOGT.

La configuración inicial del CI también requirió más tiempo del previsto, porque se tuvo que separar la aplicación Express del arranque del servidor, preparar datos ficticios y estabilizar PostgreSQL en GitHub Actions y hay que investigar el límite de uso de GitHub Actions en el plan actual para evitar que el CI consuma el tiempo disponible antes de terminar el proyecto.

Conclusiones

- El Sprint 9 se completó con los 38 story points comprometidos, alcanzando un 100 % de cumplimiento y una nota estimada de 9.6/10. El burndown llegó a cero, aunque el cierre se concentró en las últimas 36 horas, lo que evidencia una buena planificación pero una mejorable distribución del trabajo.

- Se entregaron funcionalidades como el catálogo de exámenes, el QR endurecido, el historial de precios, la bitácora de expedientes, la búsqueda tolerante con Damerau-Levenshtein y el resumen acumulado de ventas por producto. Estas mejoras fueron bien recibidas por el product owner, especialmente el historial de ventas y el de precios.

- La integración continua con GitHub Actions quedó funcional y ejecuta pruebas de backend, frontend, build e integración con PostgreSQL en cada push y pull request hacia develop.

- En refactorización, VentaService pasó de 468 a 142 líneas, se separaron webhooks y pagos POS, y se centralizaron errores y validaciones en 15 de 30 controladores. Queda pendiente migrar los controladores restantes y refactorizar componentes grandes del frontend para el Sprint 10.

- El principal aspecto a mejorar es distribuir las tareas de cierre durante todo el sprint y no dejarlas para los últimos días. También se debe revisar el CI después de cada push y controlar el consumo de GitHub Actions. El proyecto está al 90 % del tiempo y presupuesto, por lo que el último sprint debe enfocarse en estabilizar el sistema y preparar la entrega final.

Anexo

1. Formulario LOGT:

Josué García - 24918						
Fecha	**Inicio**	**Fin**	**Tiempo de interrupción**	**Delta tiempo**	**Fase**	**Comentarios**
07/10/2026	16:00	21:00	60 min	300 min	Búsqueda tolerante con Damerau-Levenshtein	Cena
08/10/2026	19:00	23:00	60 min	240 min	Refactorización de VentaService (webhooks y pagos POS)	Descanso
09/10/2026	12:00	2:00	30 min	120 min	¿Quiso decir…?" en la búsqueda del POS	Almuerzo
12/10/2026	21:00	22:00	0 min	60 min	Despliegue y pruebas en producción	Ninguno

Pablo Orellana - 20555						
Fecha	**Inicio**	**Fin**	**Tiempo de interrupción**	**Delta tiempo**	**Fase**	**Comentarios**
09/10/2026	13:00	18:30	30	300 min	arias de frontend/backend y regresion de las refa	N/A
12/10/2026	17:00	20:00	0	210 min	Consolidación de correos y teléfonos	N/A

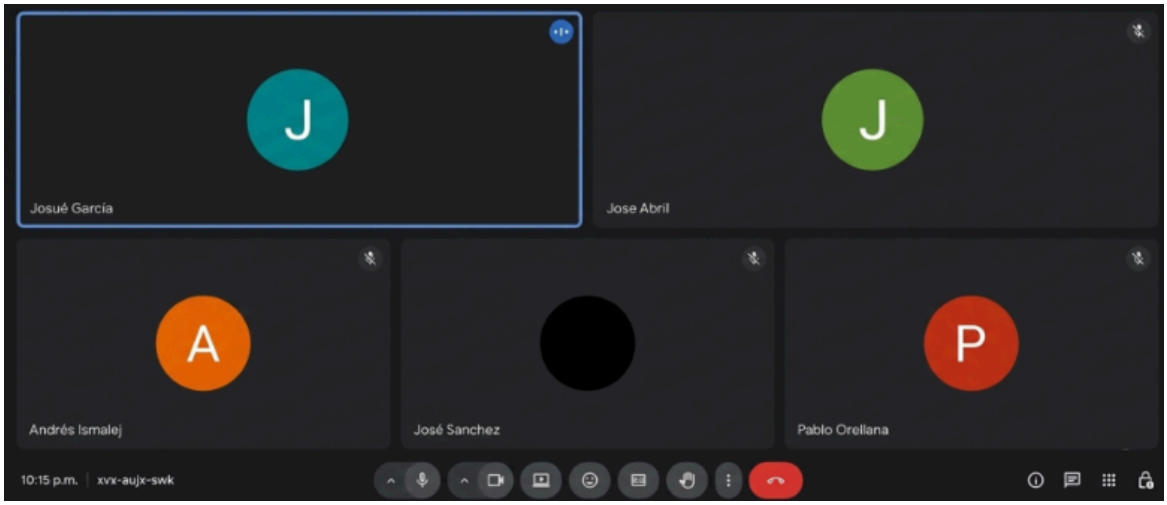
José Sanchez - 24092						
Fecha	**Inicio**	**Fin**	**Tiempo de interrupción**	**Delta tiempo**	**Fase**	**Comentarios**
03/10/2026	18:00	19:00	0 min	60 min	Creación de tareas en Jira	Ninguno
05/10/2026	20:00	22:00	0 min	120 min	Base de CI	Ninguno
06/10/2026	19:30	20:30	0 min	60 min	Pruebas de integración para CI	Ninguno
10/10/2026	10:00	12:00	30 min	90 min	Centralización de manejo de errores en backend	Fui a comer
10/10/2026	18:00	19:00	0 min	60 min	Resumen de ventas por producto en ventas	Ninguno
10/10/2026	19:00	21:00	30 min	90 min	Vista de historial y resumen de ventas	Fui a cenar

Jose Abril - 24585						
Fecha	**Inicio**	**Fin**	**Tiempo de interrupción**	**Delta tiempo**	**Fase**	**Comentarios**
04/10/2026	20:00	21:00	0 min	60 min	Base de datos: historial de cambios de precio	Ninguno
04/10/2026	21:00	23:00	0 min	120 min	Backend: historial de cambios de precio	Ninguno
05/10/2026	10:00	12:00	30 min	90 min	Backend: historial de cambios del expediente	Ninguno
05/10/2026	12:00	14:00	0 min	120 min	Frontend: historial de cambios del expediente	Ninguno
06/10/2026	10:00	13:00	90 min	90 min	Refactorizacion	Ninguno

Andrés Ismalej - 24005						
Fecha	**Inicio**	**Fin**	**Tiempo de interrupción**	**Delta tiempo**	**Fase**	**Comentarios**
07/10/2026	19:00	21:00	0 min	120 min	Consolidación de QR	Ninguno
13/10/2026	10:00	11:00	0 min	60 min	Documentación del Sprint	Ninguno

Enlace a los formularios LOGT:
https://docs.google.com/spreadsheets/d/1ypNj7HF7U50eDfjyMDa7rLbJeJHUuNMgyNzShaK8cP8/edit?usp=sharing

2. Reuniones del equipo:


J
Josué García
J
Jose Abril
A
Andrés Ismalej
José Sanchez
P
Pablo Orellana
10:15 p.m. | xvx-aujx-swk